

Electron 子窗口管理

前言

BrowserWindow 创建的子窗口，需要创建独立的渲染进程。

通过 window.open 打开的子窗口 不会创建新的渲染进程，依赖于主进程的渲染进程。

定义独立的 Partition, 在关闭子窗口时会释放资源内存。

版本

- **Electron version 38.1.0**
- **vue version 3.5**
- **vue router version 4.5.1**

渲染层

假设创建: `/#/setting`, `/#/about`

第一个参数: 传递已创建的路由后缀 `/setting` or `/about`

第二个参数传递: `'_blank'`, 即使窗口已窗口也会再次调用。

第三参数传递窗口配置项 和 扩展窗口信息

```
import { UrlUtils } from './UrlUtils';
```

```
export const customApi = {
```

```
  /**
```

```
   * 打开窗口 window.open
```

```

*
* @param routeUrl 路由地址
* @param params    参数
* @param pathParams 路径参数
* @param winOptions 窗口配置项 和 扩展窗口信息
*/
openWindow: (
  routeUrl: string,
  {
    params = {},
    pathParams = {},
    ..options
  }
) => {
  // 拼接 URL
  const suffixUrl = UrlUtils.buildUrl(routeUrl, {
    pathParams,
    params,
  });
  return window.open(
    `#${suffixUrl}`,
    '_blank',
    JSON.stringify({
      ...options,
    })
  );
},
};

```

主进程

其他处理逻辑需要补充的内容

1. 窗口复用处理: 由于 window.open 第二个参数传递了 `_blank` , 每次 window.open 调用都会执行一次 `setWindowOpenHandler` , 方便于后续处理窗口的显示和隐藏处理。
2. 扩展缓存机制: 全局定义定时器 每隔 N 秒执行一次 遍历检查所有窗口是否过期。
3. 子窗口定义 `closed` 事件, 如果子窗口处于 `缓存中` 就隐藏窗口, 否则销毁关闭窗口。
4. 如果子窗口中需要打开其他子窗口, 也同理需要实现 `setWindowOpenHandler`

```

// 窗口缓存
const windowMap = new Map<string, OpenWindowOption>();

/**

```

```

* 处理 window.open 请求
*
* @param details - window.open
*/
const handleOpenRequest = (
  details: HandlerDetails
) => {
  // returnUrl (如 "#/setting")
  const returnUrl = details.url.replace(details.referrer.url, '');

  // 传递过来的 winOptions
  const {
    cacheExpired,
    ...options,
  } = details.features
    ? JSON.parse(details.features)
    : {};

  // 检查缓存, 复用已存在的窗口
  const cachedInfo = windowMap.get(returnUrl);
  //...其他处理逻辑

  const windowOptions: Electron.BrowserWindowConstructorOptions = {
    //...其他配置项
    webPreferences: {
      preload: join(__dirname, '../preload/index.js'),
      sandbox: true,
      spellcheck: false,
      // 独立的 partition
      partition: `persist:bluereba-chat-${returnUrl}`,
    },
    ...options,
  };

  // 监听页面加载完成事件
  app.once('browser-window-created', (_, win) => {
    win.webContents.once('did-finish-load', () => {
      const url = win.webContents.getURL();
      const hashIndex = url.indexOf('#');
      const returnUrl = hashIndex !== -1 ? url.slice(hashIndex) : url;

      // 存储窗口信息
      loadWindowInfo(returnUrl, win, {
        // 定义缓存时间
        cacheExpired: cacheExpired || 10 * 1000,
        ...options,
      });
    });
  });
};

```

```

return {
  action: 'allow',
  overrideBrowserWindowOptions: {
    ...windowOptions,
    webPreferences: {
      ...windowOptions.webPreferences,
    },
  },
};
};

/**
 * 加载窗口信息
 */
const loadWindowInfo = (
  routeUrl: string,
  win: BrowserWindow,
  options: Partial<OpenWindowOption> = {}
) => {
  // 其他处理逻辑
  windowMap.set(routeUrl, {
    id: win.id,
    cacheExpired,
    // ...其他信息
  });
};

mainWindow.webContents.setWindowOpenHandler((details) => {
  return handleOpenRequest(details);
});

```

标签: Electron vue-router